Integrante: Cristian Vasco

Desarrollo de Web Scraping sobre Instagram

EXPLICACIÓN GENERAL DEL CÓDIGO

En general este código sirve para automatizar la recolección de datos desde Instagram. Lo hice usando Playwright porque permite simular un navegador real, las expresiones regulares ayudan a encontrar datos específicos dentro del HTML, mientras que requests permite descargar archivos multimedia.

Este código se hizo usando Python. Básicamente el programa entra al perfil, busca algunas publicaciones, abre cada post y obtiene datos como el texto del post, cantidad de likes, comentarios, fecha y también descarga las imágenes o videos. Al final guarda toda esa información en archivos JSON y CSV.

El programa funciona usando varias librerías que ayudan a hacer distintas tareas.

Como primer paso se extrajo las cockies con la extensión "Get cookies.txt LOCALLY", la cual nos permite exportar las cookies en un txt. Y el cual lo vamos a colocar en la carpeta del proyecto

cookies.txt

```
# Netscape HTTP Cookie File
# https://curl.haxx.se/rfc/cookie_spec.html
# This is a generated file! Do not edit.

.instagram.com	TRUE	/	TRUE	1810956685	datr	janhaZ6wXx2tcnMe2mQQbIpK
.instagram.com	TRUE	/	TRUE	1807932685	ig_did	336997A4-65EC-4921-AAAF-B332AF8F5D50
.instagram.com	TRUE	/	TRUE	1810956685	mid	aeGpjQALAAGmgAz71rtNvElr8BR9
.instagram.com	TRUE	/	TRUE	1810956686	ps_l	1
.instagram.com	TRUE	/	TRUE	1810956686	ps_n	1
.instagram.com	TRUE	/	TRUE	1807932708	ig_nrcb	1
.instagram.com	TRUE	/	TRUE	1807932710	fbm_124024574287414	base_domain=.instagram.com
.instagram.com	TRUE	/	TRUE	1811131934	csrftoken	7CGf6UprIDHzJs8uAnmyrTFvQNTMXtpx
.instagram.com	TRUE	/	TRUE	1784347934	ds_user_id	1649269179
.instagram.com	TRUE	/	TRUE	1808104927	sessionid	1649269179%3AZZ4Q41eAXePvh0%3A6%3AAYidL0tNZeCQE
.instagram.com	TRUE	/	TRUE	1777176722	dpr	0.3333333432674408
.instagram.com	TRUE	/	TRUE	1777176724	wd	1366x633
.instagram.com	TRUE	/	TRUE	0	rur	"RVA\0541649269179\0541808107934:01fe610252ea273073c824c15de5a1
```

Como siguiente paso ejecutamos el script "convert_cookies.py" la cual toma el txt antes realizado y convierte sus datos a un json que es más fácil de usar en scripts, APIs o automatizaciones Y como consiguiente nos da el archivo json que vamos a utilizar en el scraping.

convert_cookies.py > convertir_cookies

```python
import json

Qodo: Test this function
def convertir_cookies():

    cookies = []

    with open("cookies.txt", "r", encoding="utf-8") as f:

        for line in f:

            if line.startswith("#"):
                continue

            parts = line.strip().split("\t")

            if len(parts) >= 7:

                cookie = {
                    "domain": parts[0],
                    "path": parts[2],
                    "secure": parts[3] == "TRUE",
                    "name": parts[5],
                    "value": parts[6]
                }

                cookies.append(cookie)

    with open("cookies.json", "w") as f:
        json.dump(cookies, f, indent=2)

    print("✅ Cookies convertidas correctamente")


convertir_cookies()
```

cookies.json > {} 0

```
[
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "datr",
    "value": "janhaZ6wXx2tcnMe2mQQbIpK"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "ig_did",
    "value": "336997A4-65EC-4921-AAAF-B332AF8F5D50"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "mid",
    "value": "aeGpjQALAAGmgAz71rtNvElr8BR9"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "ps_l",
    "value": "1"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "ps_n",
    "value": "1"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "ig_nrcb",
    "value": "1"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
    "name": "fbm_124024574287414",
    "value": "base_domain=.instagram.com"
  },
  {
    "domain": ".instagram.com",
    "path": "/",
    "secure": true,
```

Luego de haber creado nuestro json de las cookies continuamos con la ejecución del script principal "scraper_pro.py", donde a continuación explicamos el código:

- **Primero** importamos las librerías necesarias:

```
scraper_pro.py > ...
import asyncio
import json
import os
import requests
import re
import csv
from datetime import datetime

from playwright.async_api import async_playwright

```

La librería **asyncio** se usa para ejecutar funciones asíncronas. Esto es necesario porque Playwright trabaja con funciones async, lo que permite que el programa haga varias cosas sin quedarse bloqueado esperando.

La librería **json** se usa para leer y guardar archivos en formato JSON. En este caso se usa para cargar las cookies desde un archivo llamado cookies.json y también para guardar los resultados finales en un archivo posts.json.

- La librería **os** sirve para trabajar con carpetas y archivos. Se usa para crear las carpetas donde se guardarán los datos descargados, como imágenes y videos.

  La librería **requests** sirve para descargar archivos desde internet. En este código se usa para descargar las imágenes o videos que se encuentran en los posts.

  La librería **re** sirve para trabajar con expresiones regulares. Esto permite buscar partes específicas dentro del HTML de la página, como por ejemplo el caption, likes o comentarios.

  La librería **csv** se usa para crear archivos CSV. Esto permite guardar los datos en forma de tabla para poder abrirlos después en Excel.

  La librería **datetime** sirve para convertir números de tiempo en fechas normales que se puedan leer.

  Por último, se usa **Playwright**, que es la librería principal. Esta librería permite abrir un navegador automático, entrar a páginas web, hacer scroll y obtener el contenido HTML.

- **Segundo:** las configuraciones

```
# ========================
# CONFIG
# ========================

USERNAME = "latriecu"
CANTIDAD_POSTS = 5

COOKIES_FILE = "cookies.json"

DATA_DIR = "instagram_data"
IMG_DIR = f"{DATA_DIR}/media"

CSV_PATH = f"{DATA_DIR}/posts.csv"

os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(IMG_DIR, exist_ok=True)
```

En esta parte se definen algunas variables importantes.

Primero está la variable USERNAME, que indica el perfil que se va a analizar. En este caso es "latriecu".

Después está CANTIDAD_POSTS, que indica cuántos posts se quieren analizar. En este código está en 10, o sea que solo analizará los primeros 10 posts.

También se define el archivo COOKIES_FILE, que contiene las cookies necesarias para entrar a Instagram sin tener que iniciar sesión manualmente.

Luego se definen las carpetas donde se guardarán los datos. DATA_DIR es la carpeta principal y IMG_DIR es la carpeta donde se guardarán las imágenes o videos descargados.

También se define la ruta del archivo CSV donde se guardarán los datos. Las carpetas se crean automáticamente si no existen.

- **Tercero:** Función de descarga de archivos

```python
# ========================
# DESCARGA
# ========================

Qodo: Test this function
def descargar_archivo(url, filename):

    try:

        headers = {
            "User-Agent": "Mozilla/5.0"
        }

        r = requests.get(
            url,
            headers=headers,
            timeout=30
        )

        if r.status_code == 200:

            with open(filename, "wb") as f:
                f.write(r.content)

            return True

    except Exception as e:

        print("⚠️ Error descarga:", e)

    return False
```

Esta función sirve para descargar imágenes o videos desde una URL.

Primero se define un User-Agent para que el servidor piense que la solicitud viene de un navegador real. Esto ayuda a evitar bloqueos.

Luego se hace la solicitud a la URL usando requests.get y con 30 segundos de espera.

Si la descarga funciona correctamente, se guarda el archivo en la carpeta indicada.

Si ocurre algún error, se muestra un mensaje indicando que hubo un problema en la descarga.

- **Cuarto:** Función para carga de comentarios

```python
# ========================
# CARGAR MÁS COMENTARIOS
# ========================

Qodo: Test this function
async def cargar_mas_comentarios(page):

    try:

        for _ in range(5):

            botones = page.locator(
                "text=Ver más comentarios"
            )

            if await botones.count() > 0:

                await botones.first.click()
                            (function) wait_for_timeout: Any
                await page.wait_for_timeout(2000)

            else:
                break

    except:
        pass
```

Esta función se usa para hacer clic en el botón "Ver más comentarios" dentro de un post.

Se intenta hacer clic varias veces para que se carguen más comentarios.

Si ya no aparece el botón, el programa deja de intentar.

Esto permite que el programa obtenga más comentarios antes de analizarlos. O si ya se muestran los 5 comentarios ya no sigue e igual controlamos los tiempos para que no nos detecte como un bot.

- **Quinto:** Función para obtener imágenes y video

```python
# ========================
# EXTRAER MEDIA
# ========================

Qodo: Test this function
async def obtener_media_real(page):

    media_urls = []

    await page.wait_for_timeout(3000)

    html = await page.content()

    try:

        # VIDEO

        videos = re.findall(
            r'"video_versions".*?"url":"(https:[^"]+)"',
            html,
            re.DOTALL
        )

        if videos:

            for v in videos:

                v = v.replace("\\/", "/")

                if v not in media_urls:
                    media_urls.append(v)

            return media_urls

        carousel = re.findall(
            r'"carousel_media":\[(.*?)\]',
            html,
            re.DOTALL
        )
```

```python
    if carousel:

        urls = re.findall(
            r'"url":"(https:[^"]+)"',
            carousel[0]
        )

        for u in urls:

            u = u.replace("\\/", "/")

            if u not in media_urls:
                media_urls.append(u)

        return media_urls

    simple_imgs = re.findall(
        r'"image_versions2".*?"url":"(https:[^"]+)"',
        html,
        re.DOTALL
    )

    for url in simple_imgs:

        url = url.replace("\\/", "/")

        if url not in media_urls:

            media_urls.append(url)
            break

except Exception as e:

    print("⚠ Error media:", e)

return media_urls
```

Esta función sirve para obtener las URLs reales de las imágenes o videos que tiene un post.

Primero espera unos segundos para que la página cargue bien y luego obtiene todo el **HTML del post** usando page.content(). Ese HTML contiene los enlaces donde están las imágenes o videos.

Después busca primero si el post tiene **video**, usando expresiones regulares para encontrar la URL del video dentro del HTML. Si encuentra videos, guarda sus enlaces en una lista.

Si no encuentra videos, entonces revisa si el post es un **carrusel**, es decir, cuando hay varias imágenes en un mismo post. Si encuentra varias URLs, las guarda en la lista.

Si tampoco encuentra carrusel, busca una **imagen simple**, que es cuando el post tiene solo una imagen. Cada URL encontrada se limpia para que quede en formato correcto y también se verifica que no se repita.

Al final, la función devuelve una lista con todas las URLs encontradas, que luego se usan para descargar las imágenes o videos del post.

- **Sexto:** Extraer texto del post

```python
# ========================
# EXTRAER TEXTO BASICO
# ========================

Qodo: Test this function
def parsear_texto(html):

    caption = "NA"
    likes = 0
    comentarios = 0
    fecha = "NA"

    try:

        cap = re.search(
            r'"caption":\{"text":"(.*?)"',
            html,
            re.DOTALL
        )

        if cap:

            caption = cap.group(1)
            caption = caption.replace("\\n", " ")


        like = re.search(
            r'"like_count":(\d+)',
            html
        )

        if like:
            likes = int(like.group(1))


        com = re.search(
            r'"comment_count":(\d+)',
            html
        )
```

```python
    if com:
        comentarios = int(com.group(1))


    date_match = re.search(
        r'"taken_at":(\d+)',
        html
    )

    if date_match:

        timestamp = int(date_match.group(1))

        fecha = datetime.fromtimestamp(
            timestamp
        ).strftime("%Y-%m-%d %H:%M:%S")

except Exception as e:

    print("⚠ Error parse texto:", e)

return (
    caption,
    likes,
    comentarios,
    fecha
)
```

Esta función sirve para extraer la información básica del post, como el texto, likes, comentarios y la fecha.

Primero recibe el HTML del post, que contiene toda la información interna de la publicación.

Luego usa expresiones regulares para buscar el **caption**, que es el texto que aparece en la descripción del post.

Después busca la cantidad de **likes**, encontrando el número que aparece en el HTML.

Luego busca la fecha del post, que aparece como un número llamado timestamp. Ese número se convierte en una fecha normal usando la librería datetime para que sea más fácil de leer.

Al final, la función devuelve los datos encontrados: caption, likes, comentarios y fecha, para que puedan usarse después en el resto del programa.

- **Séptimo:** obtener los comentarios

```python
# ========================
# EXTRAER COMENTARIOS REALES
# ========================

Qodo: Test this function
async def obtener_comentarios_reales(page, caption):

    comentarios = []

    try:

        await page.wait_for_timeout(3000)

        html = await page.content()

        bloques = re.findall(
            r'"username":"(.*?)".*?"text":"(.*?)"',
            html,
            re.DOTALL
        )

        for username, texto in bloques:

            try:

                texto = texto.replace("\\n", " ")

                texto = texto.encode(
                    "utf-16",
                    "surrogatepass"
                ).decode(
                    "utf-16",
                    "ignore"
                )

                texto = texto.strip()

                if texto == caption:
                    continue

                if username.lower() == USERNAME.lower():
                    continue

                if len(texto) < 6:
                    continue

                comentario_final = f"{username}: {texto}"

                if comentario_final not in comentarios:

                    comentarios.append(comentario_final)

                if len(comentarios) == 5:
                    break

            except:
                continue

    except Exception as e:

        print("⚠ Error comentarios:", e)

    return comentarios
```

Esta función sirve para obtener algunos comentarios reales del post que hacen los usuarios.

Primero espera unos segundos para que los comentarios carguen bien y luego obtiene el HTML del post, donde están guardados los comentarios.

Después usa expresiones regulares para buscar los nombres de usuario y los textos de los comentarios dentro del HTML.

Luego limpia los comentarios eliminando saltos de línea y caracteres extraños para que el texto quede legible.

También se evita guardar comentarios que sean iguales al caption del post o que sean del mismo dueño del perfil, para que solo queden comentarios reales de otros usuarios.

Además, se revisa que los comentarios no se repitan y que tengan un tamaño mínimo, para evitar guardar textos muy cortos o vacíos.

Al final, la función guarda solo los primeros 5 comentarios encontrados y los devuelve en una lista, para que luego puedan guardarse junto con la información del post.

- Octavo: scroll y links

```python
# ========================
# SCROLL PERFIL
# ========================

Qodo: Test this function
async def scroll_profile(page):

    prev = 0

    while True:

        posts = page.locator("a[href*='/p/']")

        count = await posts.count()

        print("Posts visibles:", count)

        if count >= CANTIDAD_POSTS:
            break

        if count == prev:
            break

        prev = count

        await page.mouse.wheel(0, 8000)

        await page.wait_for_timeout(2500)

# ========================
# LINKS
# ========================

Qodo: Test this function
async def get_links(page):

    posts = page.locator("a[href*='/p/']")

    total = await posts.count()

    links = []

    for i in range(total):

        href = await posts.nth(i).get_attribute("href")

        if href:

            full = "https://www.instagram.com" + href

            if full not in links:
                links.append(full)

    return links[:CANTIDAD_POSTS]
```

Esta función sirve para hacer scroll automático en el perfil de Instagram para que se carguen más publicaciones.

Esto es necesario porque Instagram no muestra todos los posts de una sola vez, sino que los va cargando poco a poco cuando se hace scroll hacia abajo.

De esta forma se asegura que haya suficientes publicaciones cargadas antes de empezar a obtener los enlaces de los posts.

Primero busca en la página todos los elementos que contienen enlaces hacia publicaciones. Luego toma cada enlace encontrado y lo convierte en un enlace completo, agregando la parte principal de la URL de Instagram.

Al final, se guarda una lista con los links necesarios según la cantidad de posts definida, y esa lista se usa después para abrir cada publicación y analizarla.

- **Noveno**: analizar cada post

```python
# ========================
# SCRAP POST
# ========================

Qodo: Test this function
async def scrape_post(context, link, index):

    page = await context.new_page()

    print("\n🔗 Abriendo:", link)

    await page.goto(link)

    await page.wait_for_timeout(5000)

    # Scroll inicial

    for _ in range(4):

        await page.mouse.wheel(0, 4000)
        await page.wait_for_timeout(1500)

    # 🔥 Cargar más comentarios

    await cargar_mas_comentarios(page)

    await page.wait_for_timeout(2000)

    html = await page.content()

    (
        caption,
        likes,
        comentarios,
        fecha
    ) = parsear_texto(html)

    # MEDIA

    media_urls = await obtener_media_real(page)

    # COMENTARIOS

    comentarios_texto = await obtener_comentarios_reales(
        page,
        caption
    )

    print(f"📅 Fecha: {fecha}")
    print(f"❤️ Likes: {likes}")
    print(f"💬 Comentarios: {comentarios}")
    print(f"🔗 Media detectada: {len(media_urls)}")
    print(f"📝 Comentarios capturados: {len(comentarios_texto)}")

    media_guardada = []

    for i, url in enumerate(media_urls):

        tipo = "video" if ".mp4" in url else "image"

        ext = ".mp4" if tipo == "video" else ".jpg"

        filename = f"{IMG_DIR}/post_{index}_{i}{ext}"

        if descargar_archivo(url, filename):

            media_guardada.append(filename)

    await page.close()

    return {
        "link": link,
        "fecha": fecha,
        "caption": caption,
        "likes": likes,
        "comentarios": comentarios,
        "media": media_guardada,
        "comentarios_texto": comentarios_texto
    }
```

Esta función sirve para abrir cada publicación y obtener toda su información, por eso es una de las funciones más importantes del programa.

Primero abre el enlace del post en una nueva página del navegador y espera unos segundos para que todo el contenido cargue correctamente.

Después hace un poco de scroll dentro del post para asegurarse de que se carguen bien las imágenes, videos y comentarios. También intenta cargar más comentarios si existe el botón para ver más.

Luego obtiene el HTML del post, que contiene toda la información interna de la publicación.

Con ese HTML se llama a otras funciones que ya están hechas: una para extraer el texto del post (caption, likes, comentarios y fecha), otra para obtener las URLs de las imágenes o videos, y otra para capturar algunos comentarios reales.

Después, si se encuentran imágenes o videos, el programa los descarga y los guarda en la carpeta correspondiente.

Finalmente, toda la información del post se guarda en un

diccionario, incluyendo el link, la fecha, el texto del post, los likes, los comentarios y los archivos multimedia descargados. Esa información luego se usa para guardarla en los archivos JSON y CSV.

- Decimo: función principal

```python
# ========================
# MAIN
# ========================

Codio: Test this function
async def scrape_instagram():

    dataset = []

    async with async_playwright() as p:

        browser = await p.chromium.launch(
            headless=False
        )

        context = await browser.new_context()

        print("🍪 Cargando cookies...")

        with open(COOKIES_FILE) as f:
            cookies = json.load(f)

        await context.add_cookies(cookies)

        page = await context.new_page()

        url = f"https://www.instagram.com/{USERNAME}/"

        await page.goto(url)

        await page.wait_for_timeout(4000)

        await scroll_profile(page)

        links = await get_links(page)

        print("Links:", len(links))

        for i, link in enumerate(links):

            data = await scrape_post(
                context,
                link,
                i+1
            )

            dataset.append(data)

        await browser.close()

    with open(
        f"{DATA_DIR}/posts.json",
        "w",
        encoding="utf-8",
        errors="ignore"
    ) as f:

        json.dump(
            dataset,
            f,
            indent=4,
            ensure_ascii=False
        )

    exportar_csv(dataset)

    print("\n✅ SCRAPER COMPLETO FUNCIONANDO")


# ========================

if __name__ == "__main__":

    asyncio.run(scrape_instagram())
```

Esta función es la que controla todo el proceso del scraper, es decir, donde se ejecutan todos los pasos principales.

Primero inicia Playwright y abre un navegador automático. Luego crea un contexto donde se cargan las cookies, lo que permite entrar a Instagram sin tener que iniciar sesión manualmente.

Después abre la página del perfil de Instagram usando el nombre de usuario que se definió al inicio del programa.

Luego hace scroll en el perfil para que se carguen suficientes publicaciones y después obtiene los enlaces de los posts que se van a analizar.

Una vez que tiene los enlaces, el programa abre cada post uno por uno usando un ciclo, y llama a la función que analiza cada publicación y obtiene toda su información.

Todos los datos que se obtienen se guardan en una lista general llamada dataset.

Cuando ya termina de analizar todos los posts, el navegador se cierra y los datos guardados se exportan a un archivo JSON y también a un archivo CSV.

Al final se muestra un mensaje indicando que todo el proceso terminó correctamente.